FP8/FP4/BF16 dense GEMM

Fine-grained scaling and the UE8M0 layout in DeepGEMM

DeepSeek-AI · DeepGEMM (deck 25)

Explainer deck · Industrial Pretraining Architecture & Infra Tech Landscape (2026)

FP8 is the training format, and it is narrow

- DeepSeek-V3 pre-trained **14.8T** tokens with FP8 linear layers
- E4M3 carries **3 mantissa bits**: max 448, min normal 2^{-6}
- Every projection and expert FFN is one of these GEMMs

The tensor core's own accumulator loses precision

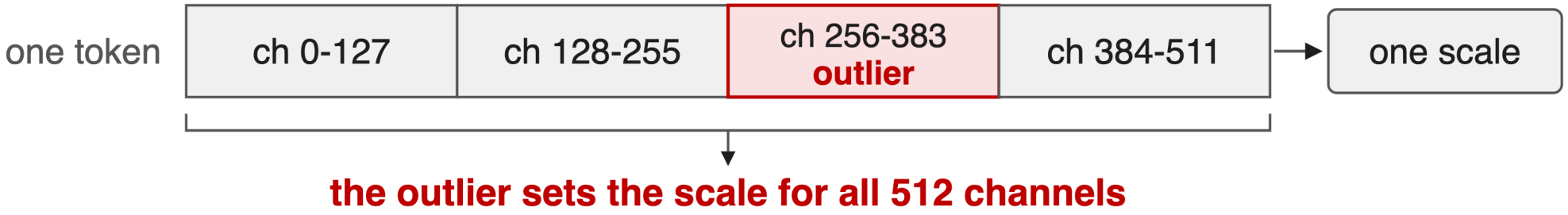
~2%

maximum relative error of an FP8 GEMM at K = 4096 — H800 tensor cores keep only about 14 bits of the accumulation

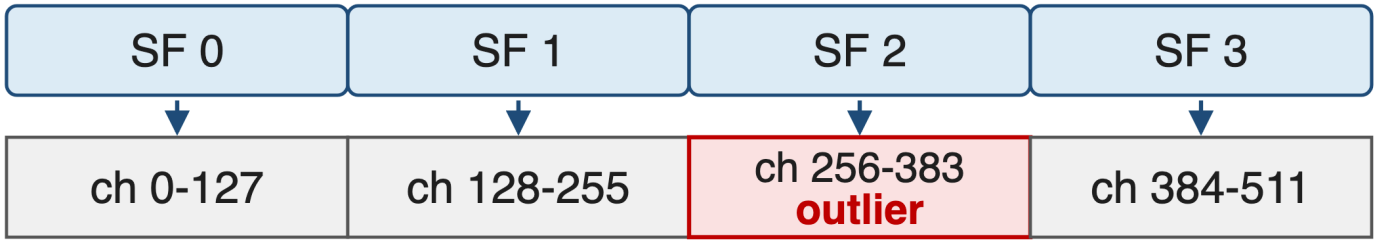
DeepSeek-V3 technical report, §3.3.2 (p. 17); error measured on two random matrices

One scale per tensor cannot survive outliers

(a) One scale per tensor



(b) One scale per 1x128 tile



SF = UE8M0 exponent
weights scale per
128x128 block

Per-group scaling along K is "not directly supported" by a standard FP8 GEMM (DeepSeek-V3 §3.3.2)

Insight: make every scale an exact power of two

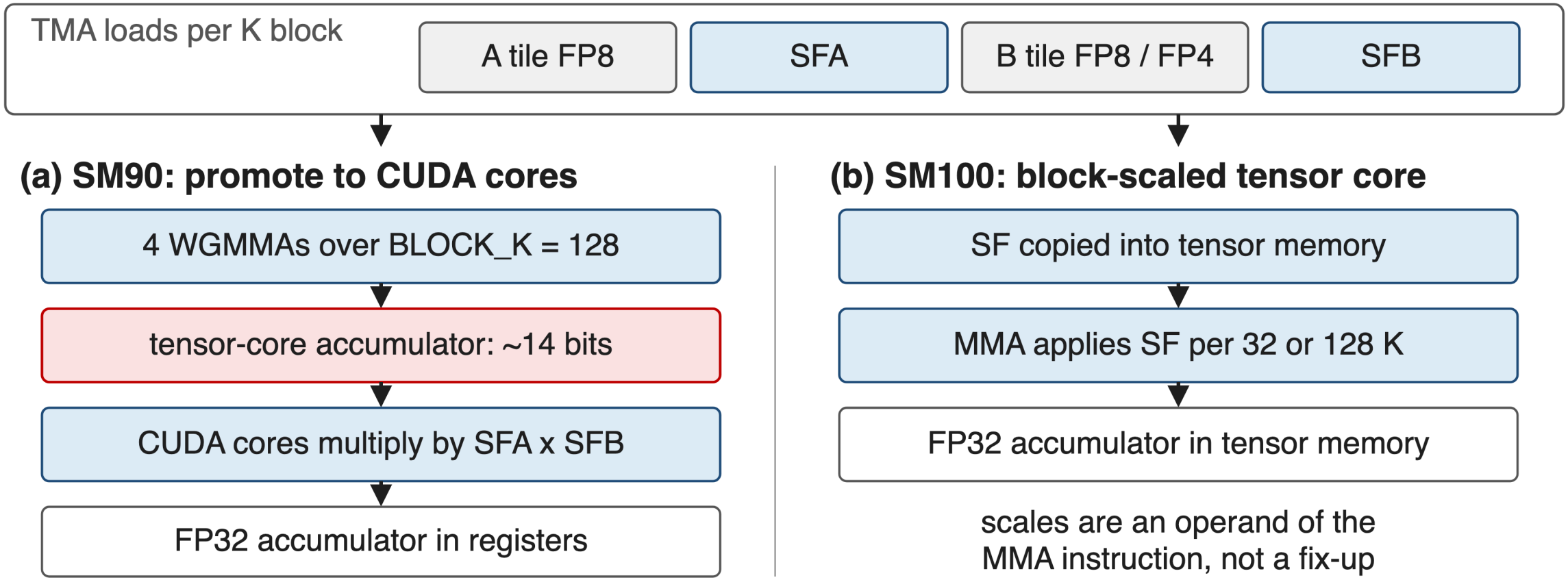
- A power-of-two scale is exactly representable and costs an exponent add
- It fits in **8 bits**: UE8M0, no sign, no mantissa

Insight: make every scale an exact power of two

- A power-of-two scale is exactly representable and costs an exponent add
- It fits in **8 bits**: UE8M0, no sign, no mantissa

Then the scale becomes an operand the TMA streams and the MMA consumes, not a fix-up outside the GEMM

Scales ride with the tiles into the inner loop



Same kernel family, two ways to apply the scale: CUDA-core promotion (SM90) or block-scaled MMA (SM100)

❶ The recipe fixes the scale-factor shapes

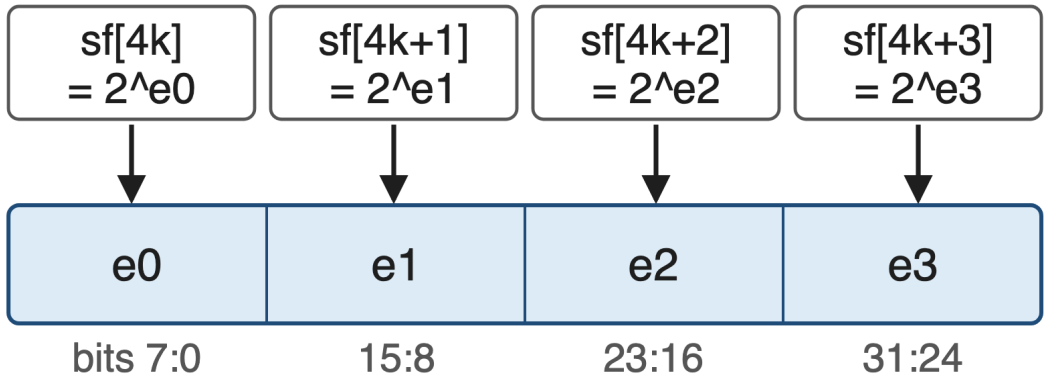
- `recipe = (gran_m, gran_n, gran_k)`: one SF per `gran_m x gran_k` block of A
- `SFA: [ceil(M/gran_m), ceil(K/gran_k)]`, FP32 before transform
- V3 uses **1x128** activations, **128x128** weights; SM100 also takes 1x32

❷ SM90 promotes to CUDA cores every 128 elements

- 4 WGMMAs fill the tensor-core accumulator, then CUDA cores apply SFA x SFB
- Partial sums land in **FP32 registers**; N_C = 128 is the minimal interval
- Two warpgroups overlap: one promotes while the other issues MMA

❸ The layout contract: packed, MN-major, aligned

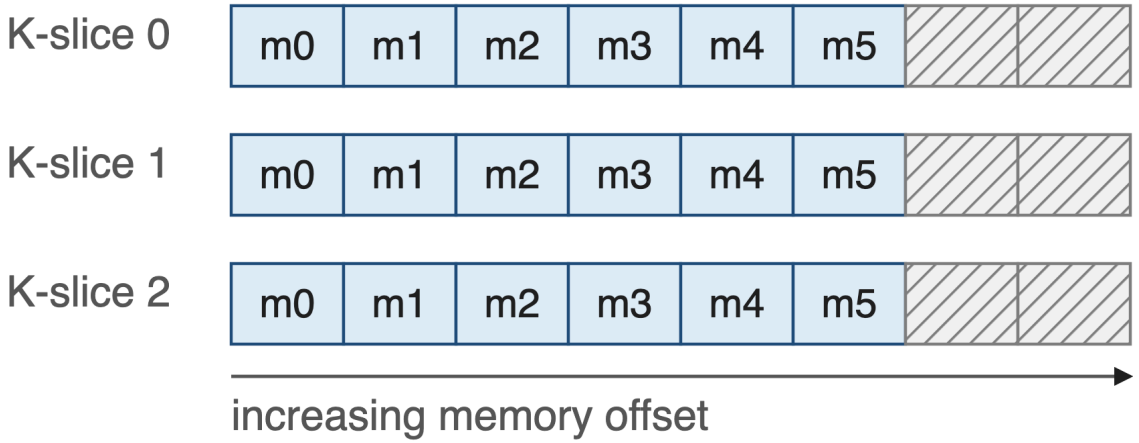
(a) Four UE8M0 exponents per int32



one int32 covers 4 x gran_k elements of K

scales must be exact powers of two

(b) MN-major with TMA padding (mn = 6)



stride(-2) = 1, stride(-1) = align(6, 4) = 8

hatched slots exist only for 16-byte TMA alignment

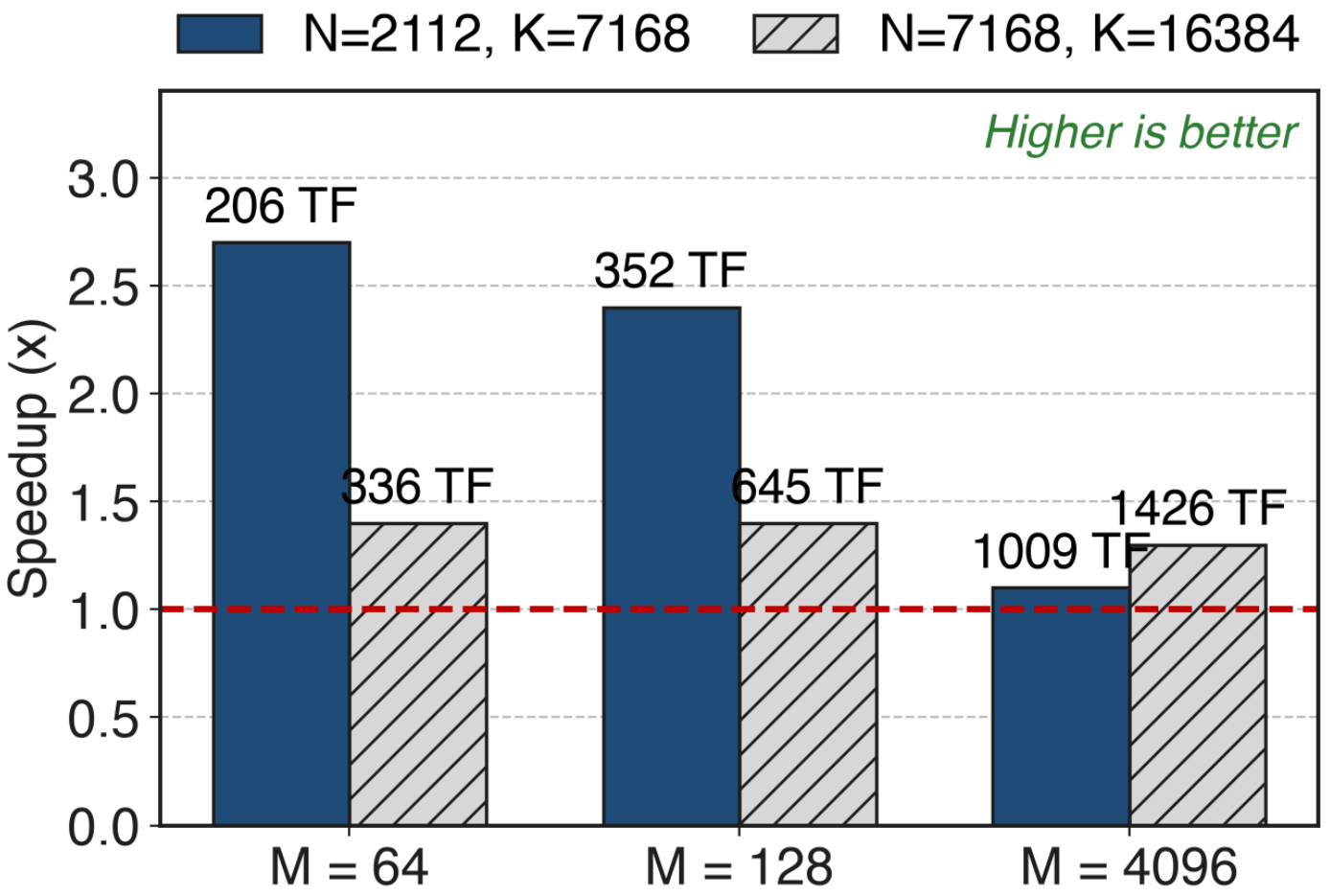
packed_sf_k = ceil(K / (gran_k x 4)); pre-transformed factors are validated, not rebuilt

FP4 and BF16 ride the same path

- FP8 x FP4 weights: 1x32 scales, BLOCK_K = 256, two values per byte
- BF16 GEMM dispatches to cuBLASLt unless determinism is requested
- Epilogues can emit FP8 directly, with per-32 UE8M0 output scales

2.7x at decode shapes, 1.3x at prefill

- Red line: tuned CUTLASS 3.6 build, H800 SXM5
- At M = 64 the kernel is bandwidth-bound at **2.6 TB/s**
- Peak across shapes: **1550 TFLOPS**



Fine-grained FP8 trains within 0.25% of BF16

<0.25%

relative training-loss error of the FP8 recipe against a BF16 baseline, two model scales, about 1T tokens

DeepSeek-V3 technical report, §3.3.1 (p. 16) and Appendix B.1

What it costs you

- SM90: NT layout only, FP32 scale factors; SM100: all majors, packed UE8M0
- Scales must be exact powers of two; unpacked ones cost a transform per call
- Casting, transposition and scale production stay in your kernels

Summary

- FP8 tensor cores keep ~**14 bits**: up to **~2%** error at K = 4096
- **Insight**: power-of-two scales fit in 8 bits and stream with the tiles
- 1x128 and 128x128 scaling, promotion on SM90, block-scaled MMA on SM100
- **2.7x** over tuned CUTLASS at decode shapes, **1550 TFLOPS** peak, **<0.25%** loss error

https://github.com/deepseek-ai/DeepGEMM